

SOFTWARE DEVELOPMENT

EXTREME PROGRAMMING AND AGILE SOFTWARE DEVELOPMENT METHODOLOGIES

Lowell Lindstrom and Ron Jeffries

Several agile (i.e., lightweight) development methodologies, especially extreme programming (XP), have been argued to be a solution to many of the problems that continue to plague software development projects. The authors provide a useful evaluation of such approaches, including a discussion of the values that underlie the XP methodology.

LOWELL

LINDSTROM has been a software professional for 20 years. He currently runs a consulting firm that helps software intensive businesses transition to more agile business processes. He can be reached at lindstrom@objectmentor.com

RON JEFFRIES is an independent

consultant. He is the senior author of Extreme Programming Installed from Addison-Wesley, and has just released a new book, Extreme Programming Adventures in C#, from Microsoft Press. He can be reached at ronjeffries@acm.org.

AS A STAKEHOLDER OF A SOFTWARE project, how does the following sound to you? You can have releases as often as you like. The small number of defects is unprecedented. All of the features in the system are the most valuable ones to your business. At anytime, you have access to complete, accurate information as the status of any feature and of the quality of the system as a whole. The team developing your project works in an energized space with constant communication about the project. You are not dependent on any one or even two programmers for the continued success of the project. If your needs change, the development team welcomes the change of direction.

As a developer of a software project, how does the following sound to you? No one estimates your tasks but you, period! You always have access to a customer to clarify details about the features you are implementing. You are free (and required) to clean up the code whenever necessary. You complete a project every two weeks. You can work in any part of the system that you wish. You can pick who

will help you on any given task. You are not required to constantly work long hours.

Does this sound too good to be true? Teams are achieving these advantages using a relatively new set of software methodologies. Collectively, they are referred to as Agile Software Development Methodologies. The most pervasive is Extreme Programming. This article introduces these exciting, popular, yet controversial new approaches to software development.

BACKGROUND

Trends in Software Development

The last half-century has seen a dizzying progression of technical advancement in the areas of computer, software, and communications technology. With each advance came rapid changes in the way society works and lives. The impact of technology is increasingly pervasive. Even as the current economic downturn limits capital investment, innovators and entrepreneurs are pushing the limits in the areas of biotechnology and nanotechnology.

The most popular project management techniques focus on developing a plan and sticking to the plan. This improves coordination but reduces the ability of the project to adapt to new information regarding the requirements or the implementation details.

Supporting all these advances is software running on some kind of computer. From the spreadsheets to anti-lock brakes to phones to toys, logic that was once implemented in mechanics, circuitry, or pencil and paper is now a set of instructions controlling one or many computers. These computers communicate with each other over networks ignorant to the limits of geography and even wires. The notion of a program running on a computer is largely a memory as the emergence of distributed computing models, most recently Web services, allows many different computers to participate in "running" a program or system to yield a given output.

In this explosive environment, software professionals have the challenge to deliver a seemingly infinite backlog of software projects, while keeping abreast of the latest advances. The results are debatable at best. Survey after survey continues to confirm that most software projects fail against some measure of success. Most software developers have many more stories of failure than success. Given the friction and finger-pointing that accompanies the end of a failed project, it is difficult to research the causes of software failures. However, typically, projects fail for one or more of the following reasons:

- ☐ Requirements that are not clearly communicated
- ☐ Requirements that do not solve the business problem
- ☐ Requirements that change prior to the completion of the project
- ☐ Software (code) that has not been tested
- ☐ Software that has not been tested as the user will use it
- ☐ Software developed such that it is difficult to modify
- ☐ Software that is used for functions for which it was not intended
- ☐ Projects not staffed with the resources required in the project plan
- ☐ Schedule and scope commitments are made prior to fully understanding the requirements or the technical risks

Despite these delivery problems, strong demand seems to hold steady. Reports still suggest a shortage of software professionals and college graduates, despite the implosion of the Internet industry slowing the growth of demand and the emergence of an offshore programming market increasing supply.

Improvement Remains Elusive

Efforts to improve the success rate of software projects have existed since the first bug was detected. Recently, these efforts have focused on four distinct parts of the software development process: requirements gathering, designing and developing the software, testing the results, and overall project management. Formal requirements definition and analysis addressed the problem of requirements that were incomplete or did not reflect the needs of the customer. Formal design before implementation addressed the goals of reuse, consistency of operation, and reducing rework. Testing caught defects before they reached the users. Project management addressed the problem of coordinating the efforts of multi-department teams. These areas of focus follow a logical approach to process improvement: improve the quality of the inputs (requirements), improve the quality of the output (designing and developing, project management), and improve the detection and elimination of defects prior to shipping (testing). Methods and tools focusing on these four distinct areas of software development have proliferated.

For some projects, these efforts have been effective. For others, they yielded silos of responsibility, with poor communication between the groups and distributed ownership and accountability. If a project is late, it is easy for the programmers to blame the requirements gatherers, and vice versa. If the users detect defects, finger-pointing ensues between developers and testers. Project management techniques try to better coordinate the activities of the multiple groups involved in delivering the project, but add yet another silo and area of accountability. The most popular project management techniques focus on developing a plan and sticking to that plan. This improves coordination but reduces the ability of the project to adapt to new information regarding the requirements or the implementation details.

The Emergence of Agile Methods

The additional process steps, roles, and artifacts helped many teams to enjoy higher success rates and more satisfied customers. Unfortunately, many projects failed attempting to use the same techniques. Some projects got lost in the documents and never implemented any code, missing the window of opportunity for the software. Others did not leave enough time at the end for implementation and testing

The set of known successes with XP continues to stretch the breadth of projects applicable for XP.

and delivered systems inconsistent with the documents and designs on which most of the project time was spent.

At the same time, numerous projects were very successful that did not follow methods with binders of documents, detailed designs, and project plans. Many experienced programmers were having great success without all these extra steps. The determining factor of project success seemed more and more to be the people on the project, not the technology or the methods that were being used. After all, people end up writing the software at some point. To some, the developers who did not embrace the new methodologies appeared to be undisciplined and indifferent to quality, despite their successes at delivering quality software that people wanted to use.

A few people started to author papers about these disciplined, yet lighter approaches to software development. They called them Extreme Programming, SCRUM, Crystal, Adaptive, etc. Different authors emphasized different aspects of software development. Some focused on approaches to planning and requirements; some focused on ways to write software that could be changed more easily; and some focused on the people interactions that allow software developers to more easily adapt to their customers' changing needs. These various efforts created a focal point for a community that furthered the set of practices that succeed without many of the activities and artifacts required by more defined methodologies.

In the fall of 1999, *Extreme Programming, Embrace Change*¹ was published and the trend had found its catalyst. In early 2001, the different innovators who were creating different agile methodologies held a retreat and scribed the "Agile Manifesto for Software Development."² By the spring of 2002, Computerworld.com ran the following headline: "More than two-thirds of all corporate IT organizations will use some form of 'agile' software development process within 18 months, Giga Information Group predicted this week at its application development conference here."³

Agile Methodologies

At the retreat in early 2001, a number of leaders of the agile software development movement held a retreat to discuss their approaches and to explore the commonalities and differences. What emerged was the Agile Manifesto for Software Development. The manifesto (see Table 1)

articulates core values and principles that guide agile methodologies.

EXTREME PROGRAMMING

Extreme Programming (XP) is the most widely used agile methodology. XP shares the values espoused by the Agile Manifesto for Software Development but goes further to specify a simple set of practices. Whereas many popular methodologies try to answer the question "What are all of the practices I might ever need on a software project?," XP simply asks, "What is the simplest set of practices I could possibly need and what do I need to do to limit my needs to those practices?"

The significance of this difference cannot be understated. The most frequent critique of XP is that it is too simple to work beyond a narrow set of project criteria. Yet, the set of known successes with XP continues to stretch the breadth of projects applicable for XP. It would seem that the parameters that we use to determine what methods are appropriate for what project are still inadequate.

To many, XP is a set of 12 interdependent software development practices. Used together, these practices have had much success, initially with small teams, working on projects with high degrees of change. However, the more one works with XP, the more it is apparent that the practices do not capture the essence of XP. As with the heavier methods, some teams have great success with the XP practices, some less so. Some larger teams have greater success than smaller ones. Some teams with legacy code have success; others do not. There is something more than just the practices that enables teams to succeed with XP. This extra attribute of XP is XP Values.

What Is Extreme Programming?

Extreme Programming is a discipline of software development based on values of simplicity, communication, feedback, and courage. It works by bringing the whole team together in the presence of simple practices, with enough feedback to enable the team to see where they are and to tune the practices to their unique situation.

In XP, every contributor to the project is a member of the "Whole Team," a single business/development/testing team that handles all aspects of the development. Central to the team is the "Customer," one or more business representatives who sit with the team and work with them daily.

TABLE 1 Agile Manifesto

We are uncovering better ways of developing software by doing it and helping others to do it. Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value on the items on the right, we value the items on the left more.

Principles behind the Agile Manifesto

We follow these principles:

- Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- Welcoming changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
- Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to shorter time scales.
- Business people and developers must work together daily throughout the project.
- Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
- The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
- Working software is the primary measure of progress.
- Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- Continuous attention to technical excellence and good design enhances agility.
- Simplicity — the art of maximizing the amount of work not done — is essential.
- The best architectures, requirements, and designs emerge from self-organized teams.
- At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Agile Methods

The number of methods that claim to align the Agile Manifesto will continue to grow with the popularity of the agile software methodologies. The early initial methodologies include:*

- Extreme Programming
- SCRUM
- Crystal
- Feature Driven Development
- Lean Development
- Adaptive Software Development
- DSDM

* Jim Highsmith, *Agile Software Development Ecosystems*, Addison-Wesley, 2002, provides a comparison of these methodologies.

XP teams use a simple form of planning and tracking to decide what to do next and to predict when any desired feature set will be delivered. Focused on business value, the team produces the software in a series of small, fully integrated releases that pass all the tests that the Customer has defined. The core XP practices for the above are called Whole Team, Planning Game, Small Releases, and Acceptance Tests. There are specific recommendations for all of these, which are briefly discussed here and as the chapter progresses.

Extreme Programmers work together in pairs and as a group, with simple design and obsessively tested code, improving the design continually to keep it always just right for the current needs.

The core XP practices here are Pair Programming, Simple Design, Test-Driven Development, and Design Improvement.

The XP team keeps the system integrated and running all the time. The programmers write all production code in pairs, and all work together all the time. They code in a consistent style so that everyone can understand and

TABLE 2 Comparison of Methodologies

Methodology ^a	Values	Principles	Practices
CMMI	No	No	Yes (KPA's)
SA/SD	No	No	Yes
RUP	No	Yes	Yes
Agile	Yes	Yes	No
XP	Yes	Yes	Yes

^a CMMI: Capability Maturity Model Integration, Software Engineering Institute.

SA/SD: Structured Analysis/Structured Design.

RUP: Rational Unified Process, Rational Corporation.

improve all the code as needed. The additional practices here are called Continuous Integration, Team Code Ownership, and Coding Standard.

The XP team shares a common and simple picture of what the system looks like. Everyone works at a pace that can be sustained indefinitely. These practices are called Metaphor and Sustainable Pace.

XP Values

The XP Values are Communication, Simplicity, Feedback, and Courage.

The essence [of XP] truly is simple. Be together with your customer and fellow programmers, and talk to each other. Use simple design and programming practices, and simple methods of planning, tracking, and reporting. Test your program and your practices, using feedback to steer the project. Working together this way gives the team courage.⁴

These values guide our actions on the project. The practices leverage these values to remove complexity from the process. The impact of the XP Values is significant and unique. XP remains the only methodology that is explicit in its values and practices. This combination gives specific guidance not only on what (the practices) to do on a project, but also on how to react (defer to the values) when the practices do not seem to be working or are not sufficient. Most methods are specific on practices, some specify principles, but few combine both.⁵ For example, CMMI describes Key Practice Areas (KPAs) but does not articulate a set of values or principles. RUP provides guiding principles, such as Develop Iteratively, but does not include values that give guidance beyond the software development practices.

How these values are used to guide the team in its use of the practices is described later in the section "Fitting XP to Your Project." See Table 2 for a comparison of several well-known methodologies.

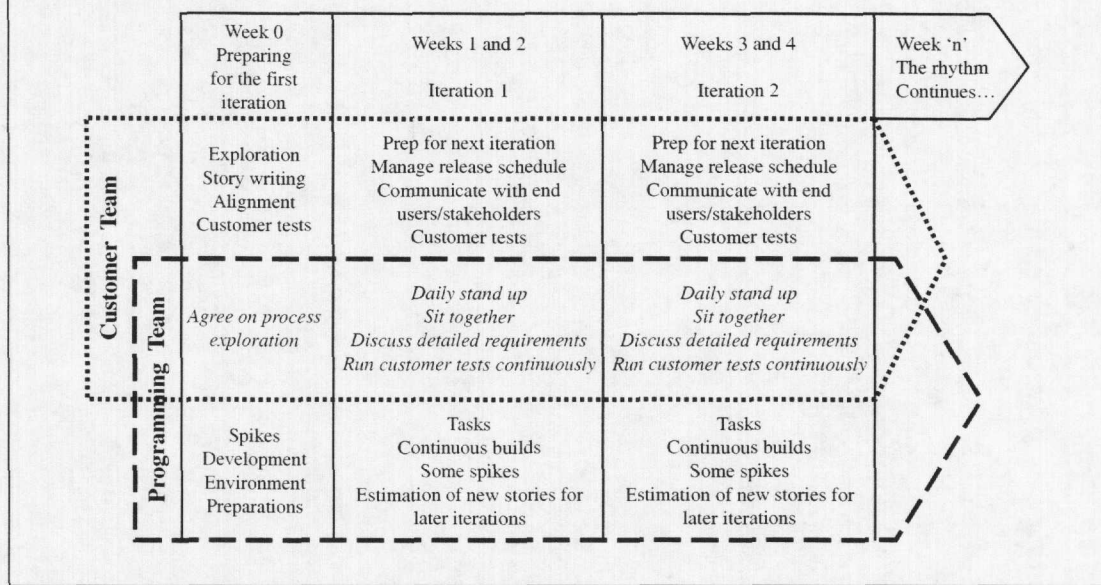
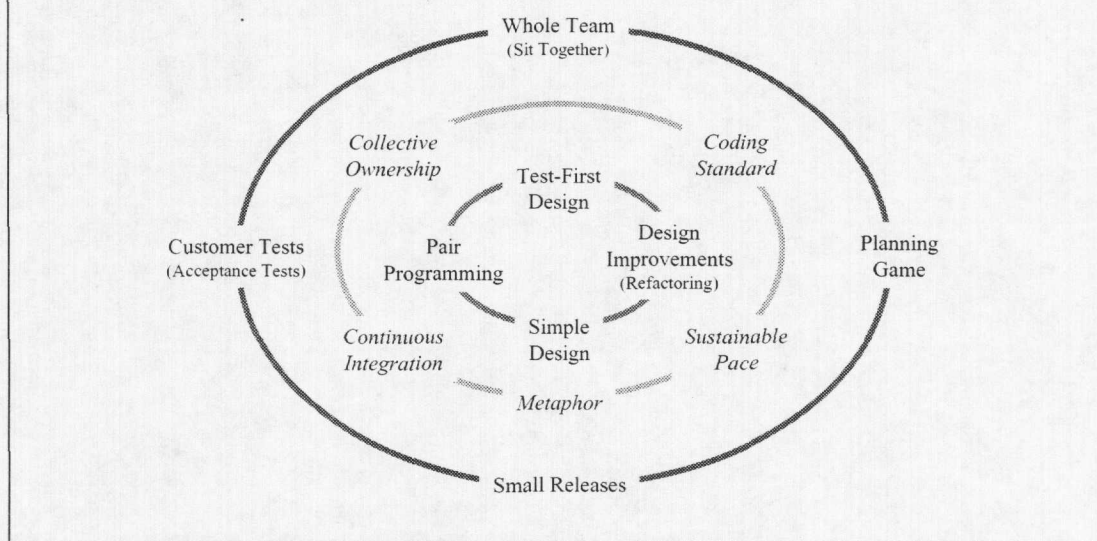
Organization

On a project using XP, there are two explicit roles or teams defined: the Customer and the Programmer. In keeping with the value of simplicity, most of the XP literature describes the customer as a single person who can represent the requirements, acceptance criteria, and business value for the project. In practice, it is a team of people that communicates with one voice with the Programming Team. As such, this role is also referred to as the Customer Team. This chapter uses the term "Customer" to describe the role, whether acted on by an individual or a team. The Programmer is a member of the Programming Team that implements the XP Customer Team's requirements. Again, the convention will be to use the term "Programmer" to describe an individual or the team.

On all but the smallest projects, there will also be a Management Team that allocates resources for the teams, manages the alignment of the project to the goals of the business, and removes any obstacles impeding the team's progress. Extreme Programming does not specify management practices. XP attempts to simplify management by empowering the Customer and Programmer to make most of the decisions regarding the project. Often, XP teams are described as self-managing. As projects grow in size and complexity, more management is typically required to coordinate the efforts of different teams. Many of the other emerging agile methodologies are focusing more attention on management practices, such as Scrum,⁶ Lean Development,⁷ and Extreme Project Management.⁸

The Rhythm of an XP Project

An XP project proceeds in iterations of two weeks in length. Each iteration delivers fully developed and tested software that meets the most valuable small set of the full project's requirements. Figure 1 shows the primary activities of the Customer and Programmer during the initial iterations of a project. The project proceeds in a steady rhythm of delivering more functionality. The Customer determines at what point in time the full system can be released and deployed.

FIGURE 1 The Rhythm of an XP Project**FIGURE 2** XP Practices and the Circle of Life**Core Practices**

There are 12 core practices that define XP. Teams new to XP should focus on using and developing skills with these practices. Over time, as the team matures in its use of XP, it will continue to check its proficiency with these practices, but will also tailor the practices to the project needs. XP teams are encouraged to use feedback from their project to adapt, add, and eliminate practices as needed. A number of

other practices are popular on XP teams and some of these are described later.

The practices can be described as a cycle of activities (see Figure 2). The inner circle describes the tight cycle of the Programmers. The outer loop describes the planning cycle that occurs between the Customers and Programmers. The middle loop shows practices that help the team communicate and coordinate the delivery of quality software.

The best XP teams treat their customer tests the same way they do programmer tests: once the test runs, the team keeps it running correctly thereafter.

Whole Team. All the contributors to an XP project sit together as members of one team. This team must include a business representative — the Customer — who provides the requirements, sets the priorities, and steers the project. It is best if the Customer or one of her aides is a real end user who knows the domain and what is needed. The team will, of course, have programmers. The team will typically include testers, who help the Customer define the customer acceptance tests. Analysts may serve as helpers to the Customer, helping to define the requirements. There is commonly a coach who helps the team stay on track and facilitates the process. There may be a manager, providing resources, handling external communication, and coordinating activities. None of these roles is necessarily the exclusive property of just one individual. Everyone on an XP team contributes in any way that he or she can. The best teams have no specialists, only general contributors with special skills.

Planning Game. XP planning addresses two key questions in software development: predicting what will be accomplished by the due date, and determining what to do next. The emphasis is on steering the project — which is quite straightforward — rather than on exact prediction of what will be needed and how long it will take — which is quite difficult. There are two key planning steps in XP:

1. *Release planning* is a practice where the Customer presents the desired features to the programmers, and the programmers estimate their difficulty. With the cost estimates in hand, and with knowledge of the importance of the features, the Customer lays out a plan for the project. Initial release plans are necessarily imprecise; neither the priorities nor the estimates are truly solid, and until the team begins to work, we will not know just how fast they will go. Even the first release plan is accurate enough for decision making, however, and XP teams revise the release plan regularly.
2. *Iteration planning* is the practice whereby the team is given direction every couple of weeks. XP teams build software in two-week “iterations,” delivering running, useful software at the end of each iteration. During Iteration Planning, the Customer presents the features desired for the next two weeks. The programmers break them down into tasks and estimate their cost (at a finer level of detail than in Release Planning).

Based on the amount of work accomplished in the previous iteration, the team signs up for what will be undertaken in the current iteration.

These planning steps are very simple yet they provide very good information and excellent steering control in the hands of the Customer. Every couple of weeks, the amount of progress is entirely visible. There is no “90 percent done” in XP: a feature story was completed, or it was not. This focus on visibility results in a nice little paradox. On the one hand, with so much visibility, the Customer is in a position to cancel the project if progress is not sufficient. On the other hand, progress is so visible, and the ability to decide what will be done next is so complete, that XP projects tend to deliver more of what is needed, with less pressure and stress.

Customer Tests. As part of presenting each desired feature, the XP Customer defines one or more automated acceptance tests to show that the feature is working. The team builds these tests and uses them to prove to themselves, and to the customers, that the feature is implemented correctly. Automation is important because in the press of time, manual tests are skipped. That is like turning off your lights when the night gets darkest.

The best XP teams treat their customer tests the same way they do programmer tests: once the test runs, the team keeps it running correctly thereafter. This means that the system only improves, always notching forward, and never backsliding.

Small Releases. XP teams practice small releases in two important ways. First, the team releases running, tested software, delivering business value chosen by the Customer, with every iteration. The Customer can use this software for any purpose, either for evaluation or even for release to end users (which is highly recommended). The most important aspect is that the software is visible, and given to the customer at the end of every iteration. This keeps everything open and tangible. Second, XP teams also release software to their end users frequently. XP Web projects release as often as daily, in-house projects monthly or more frequently. Even shrink-wrapped products are shipped as often as quarterly.

It might seem impossible to create good versions this often but XP teams are doing it all the time. See the section on “Continuous

Extreme
programming
focuses on
delivering
business
values in every
iteration.

Integration” for more on this, and note that these frequent releases are kept reliable by XP’s obsession with testing, as described in the sections on “Customer Tests” and “Test-Driven Development.”

Simple Design. XP teams build software to a simple design. They start simple, and through programmer testing and design improvement, they keep it that way. An XP team keeps the design exactly suited for the current functionality of the system. There is no wasted motion, and the software is always ready for what is next.

Design in XP is neither a one-time thing nor an up-front thing, but it is an all-the-time thing. There are design steps in release planning and iteration planning, plus teams engage in quick design sessions and design revisions through refactoring, throughout the course of the entire project. In an incremental, iterative process like Extreme Programming, good design is essential.

Pair Programming. In XP, two programmers, sitting side by side at the same machine, build all production software. This practice ensures that all production code is reviewed by at least one other programmer, resulting in better design, better testing, and better code.

It may seem inefficient to have two programmers doing “one programmer’s job,” but the reverse is true. Research on pair programming shows that pairing produces better code in about the same time as programmers working singly. That is right: two heads really are better than one!

It does take some practice to do well, and you need to do it well for a few weeks to see the results. Most programmers who learn pair programming prefer it, so we highly recommend it to all teams.

Pairing, in addition to providing better code and tests, also serves to communicate knowledge throughout the team. As pairs switch, everyone gets the benefits of everyone’s specialized knowledge. Programmers learn, their skills improve, and they become more valuable to the team and to the company. Pairing, even on its own outside of XP, is a big win for everyone.

Test-Driven Development. XP is obsessed with feedback; and in software development, good feedback requires good testing. XP teams practice “test-driven development,” working in very short cycles of adding a test, then making it work. Almost effortlessly, teams produce

code with nearly 100 percent test coverage, which is a great step forward in most shops. (If your programmers are already doing even more sophisticated testing, more power to you. Keep it up, it can only help.)

It is not enough to write tests; you have to run them. Here, too, XP is extreme. These “programmer tests,” or “unit tests,” are all collected together, and every time any programmer releases any code to the repository (and pairs typically release twice a day or more), every single one of the programmer tests must run correctly. One hundred percent, all the time! This means that programmers get immediate feedback on how they are doing. Additionally, these tests provide invaluable support as the software design is improved.

Design Improvement. XP focuses on delivering business value in every iteration. To accomplish this over the course of the whole project, the software must be well designed. The alternative would be to slow down and ultimately get stuck. So, XP uses a process of continuous design improvement called “refactoring.”⁹

The refactoring process focuses on the removal of duplication (a sure sign of poor design), and on increasing the “cohesion” of the code while lowering the “coupling.” High cohesion and low coupling have been recognized as the hallmarks of well-designed code for at least 30 years.¹⁰ The result is that XP teams start with a good, simple design, and always have a good, simple design for the software. This lets them sustain their development speed and, in fact, generally increase speed as the project goes forward.

Refactoring is, of course, strongly supported by comprehensive testing that ensures that as the design evolves, nothing is broken. Thus, the customer tests and programmer tests are a critical enabling factor. The XP practices support each other: they are stronger together than separately.

Continuous Integration. XP teams keep the system fully integrated at all times. We say that daily builds are for wimps; XP teams build multiple times per day. (One XP team of 40 people builds at least eight or ten times per day!) The benefit of this practice can be seen by thinking back on projects you may have heard about (or even been a part of), where the build process was weekly or less frequently and usually led to “integration hell,” where everything broke and no one knew why.

The specifics of the standard are not important; what is important is that all the code looks familiar, in support of collective ownership.

Infrequent integration leads to serious problems on a software project. First of all, although integration is critical to shipping good working code, the team is not practiced at it, and often it is delegated to people who are not familiar with the whole system. Second, infrequently integrated code is often — or usually — buggy code. Problems creep in at integration time that are not detected by any of the testing that takes place on a nonintegrated system. Third, a weak integration process leads to long code freezes. Code freezes mean that you have long time periods when the programmers could be working on important shippable features, but that those features must be held back. This weakens your position in the market or with your end users.

Collective Code Ownership. On an XP project, any pair of programmers can improve any code at anytime. This means that all code gets the benefit of many people's attention, which increases code quality and reduces defects. There is another important benefit as well: when code is owned by individuals, required features are often put in the wrong place as one programmer discovers that he needs a feature somewhere in code that he does not own. The owner is too busy to do it, so the programmer puts the feature in his own code, where it does not belong. This leads to ugly, hard-to-maintain code, full of duplication and with low (bad) cohesion.

Collective ownership could be a problem if people worked blindly on code they do not understand. XP avoids these problems through two key techniques: (1) the programmer tests catch mistakes, and (2) pair programming, which means that the best way to work on unfamiliar code is to pair with the expert. In addition to ensuring good modifications when needed, this practice spreads knowledge throughout the team.

Coding Standard. XP teams follow a common coding standard so that all the code in the system looks as if a single — very competent — individual wrote it. The specifics of the standard are not important; what is important is that all the code looks familiar, in support of collective ownership.

Metaphor. XP teams develop a common vision of how the program works, which we call the "metaphor." At its best, the metaphor is a simple, evocative description of how the program works, such as "this program works like a

hive of bees, going out for pollen and bringing it back to the hive" as a description for an agent-based information retrieval system.

Sometimes, a sufficiently poetic metaphor does not arise. In any case, with or without vivid imagery, XP teams use a common system of names to be sure that everyone understands how the system works and where to look to find functionality or to find the right place to put the functionality that is about to be added.

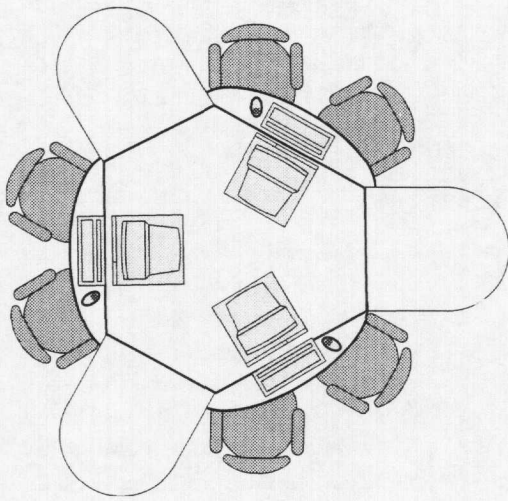
Sustainable Pace. XP teams are in it for the long term. They work hard, but at a pace that can be sustained indefinitely. This means that they work overtime when it is effective, and that they normally work in such a way as to maximize productivity week in and week out. It is well understood these days that "death march" projects are neither productive nor produce quality software. XP teams are in it to win, not to die.

Other Common Practices

The core practices of XP do not specify all of the activities that are required to deliver a software project. As teams use XP, many find that other practices aid in their success, in some cases as significantly as some of the core practices. The following are some other practices commonly used by successful XP teams.

Open Workspace. To maximize communication among the Whole Team, the team works together in an "open workspace." This is a large room, with tables in the center that can typically seat two to four pairs of developers. Figure 3 shows an example of an XP workstation for three pairs of developers. By sitting together, all team members can establish instant communication when needed for the project. Teams establish their own rules concerning their space to ensure that everyone can work effectively. The walls of the "open workspace" are used to display information about the project. This will include big, visible charts of metrics such as passing acceptance tests and team productivity. There may be designs drawn on whiteboards. Project status will be displayed so that any participant or stakeholder of the project can always see progress.

Retrospectives. The XP practices provide feedback to the team as to the quality of the code and its alignment to the Customers' needs. The team also needs feedback on how it is performing. Is it following the practices with discipline? Are there adaptations to the practices

FIGURE 3 Extreme Programming Workstation

that would benefit the team? The practice commonly used for this is the Retrospective.¹¹ After each iteration, the team does a short reflection on what went well during the iteration and what should be improved in the next iteration. After a release of the product, a more in-depth Retrospective is performed on the whole project.

Self-Directed Teams. A practice that is common among most of the agile methods is self-directed teams. The best people to make decisions about the project are those closest to the details, as long as they have an understanding of the overall goals of the project. Open communication allows team members to have the information required to make decisions. Managers are part of the communication loop but not bottlenecks in the decision-making flow.

Customer Team. As XP is used on projects with more complex requirements, a team performs the Customer function. For larger or more complex projects, the Customer team may even exceed the Programming team in size. Some of the challenges faced by the Customer team include communicating with and balancing the needs of multiple stakeholders, allocating resources to the appropriate projects or features, and providing sufficient feedback to ensure that the requirements implemented achieve the stakeholders' goals. The specific Customer team practices are still emerging in the agile community. The practices are guided by the same values as the other XP practices.

FITTING XP TO YOUR PROJECT

Is My Project a Good Fit for XP?

Probably the most commonly debated question regarding XP is whether it can be used successfully on a particular type of project. Experience is proving that, as with other approaches to software development, the limitations often include the characteristics of the project, the people on the team, and the organization in which they work. To evaluate whether the XP practices can help a team achieve greater success on their project, consideration must be given to the project characteristics, the people on the team, and the cultures of the organizations involved in the project.

The XP Values can be used as a template to test the fit of XP to a project, team, and organization. Simply evaluate the degree to which each value is currently held by the team and the organization.

Communication. Does the team communicate constantly and effectively? Does this communication extend to the customer? Is the team's software readable and understandable (i.e., is it easy for Programmers to communicate with the code)?

Simplicity. Is the team comfortable with simple solutions? Can the team implement, without a complete design, the system prior to coding? Is the team comfortable with some ambiguity as to the exact requirements and designs? Can the team adapt often to changing requirements? Is the team working new code or code that is well designed and refactored?

Feedback. Can the team get feedback on its tasks and deliverables often? Does the team accept feedback constructively? When there are problems, does the team focus on the process to identify root causes (rather than the people)? How often does the team integrate, build, and test the complete software system?

Courage. Does the organization encourage individuals to not fear failure? Are individuals and teams encouraged to show initiative and make decisions for their projects? Are organizational boundaries easily crossed to solve problems on the project?

Typically, the greater the degree to which the team can answer these questions affirmatively, the fewer changes will be required and the easier it is for the team and organization to adopt XP. Some specific project and team

guidelines for getting started are provided next.

Getting Started

When selecting an initial project on which to try XP, one must consider the challenges of using the new practices. New practices introduce risk to a project. Care must be taken to select an initial project that is not burdened by all of the most difficult obstacles to using XP, but does address enough typical obstacles so that the success of the initial project can provide the basis for expanding to the rest of the organization.

Although most initial XP projects are not this fortunate, ideally, the initial project will have many of the following characteristics:

- Primarily new code versus legacy updates
- An identified and available source of requirements and feedback (i.e., on-site customer)
- Delivers important business value and has management visibility
- Uses an OO language/environment
- Is typical of the projects the organization will be doing in the future
- Has a co-located team in an open workspace
- Can be delivered to the end user incrementally, with a new stage once in at least every four to six weeks

In selecting the initial XP Project Team, the main attribute of the team members should be a strong commitment to delivering the project and achieving its goals using the new practices. Some healthy skepticism about XP is acceptable as long as the team members are willing to use the practices and let data and experience from the project guide any adaptations. The team ideally will have a few technical leaders familiar with other projects in the organization, but it is not desirable to have a team full of the most senior people. XP is a collaborative approach to development and, as such, the initial project will benefit from members with strong "soft" skills who prefer collaborative work environments. Beyond these characteristics, the team should be representative of teams that the organization will use in the future.

The simplest way to reduce risk on an initial project is maximize the skill of the team as quickly as possible. This can be achieved through recruiting team members that are already skilled in XP, training, or experienced coaching for an inexperienced team.

Adaptations

As teams begin adopting the XP practices, numerous obstacles and constraints must be confronted. The team may have trouble gaining access to the Customer every day. The team may have trouble co-locating to an open workspace. The team may be so large that communicating without formal documentation is not feasible. How do we adjust? Must we abandon XP? The XP Values guide teams in solving these process problems with their projects.

The Courage value guides us to aggressively confront and remove any obstacles that would add steps, artifacts, or complexity to the process. This often means letting common sense outweigh bureaucracy. For example, teams sometimes do not feel empowered to change the physical work environment to have an open workspace (i.e., change the cubicles). Often, a little courage, negotiating, and a power screwdriver will remove this obstacle. Some teams struggle to have a customer sitting with the team. The programmers develop from a requirements document and have never spoken to the customer. Although the thought of having a customer present is desirable, the logistics can seem impossible, particularly if the best person to sit with the team does not live near the team or is constantly traveling. Often, with a slight reorganization and a modified communication infrastructure, a customer can be identified who can sit with the team on a frequent basis.

Of course, courage can only take us so far. There will be constraints that interfere with our ability to implement the practices as described. A common example is legacy code. Many teams work with large code bases that do not have tests and are in dire need of design improvement. We want to aggressively move to the state where all of the code has passing tests, is understandable, and is well designed. The initial attempt is to rapidly get the code up to our new standard. Can we toss it and rewrite it? Would it really be that expensive and time-consuming to fix it? Is there other, cleaner code available with which we can replace it? Very often, the answers are No, Yes, and No, respectively, leaving team members no choice but to live with the smelly code and improve as they can.

XP Values give the team a helpful, simple tool to deal with this difficult, yet inevitable challenge. The constraint that causes a practice to be modified or abandoned is reviewed against each of the XP Values, using the following question: How will the influence of this XP

Care must be taken to select an initial project that is not burdened by all of the most difficult obstacles to using XP, but does address enough typical obstacles so that the success of the initial project can provide the basis for expanding to the rest of the organization.

TABLE 3 Constraint: Legacy Code Prevents Test-Driven Development

Value	Impact	Adaptation Alternatives
Communication	Difficult to communicate the code's intent and design	• Wiki^a pages to document the team's understanding of the code • Reverse-engineering tools to create models of the code
Simplicity	Complexity restricts some simple design alternatives	• Changing pair partners at least twice a day • Targeting areas of the system that warrant simplification
Feedback	It will take longer to be aware of and fix errors	• When changes are being made in a complex area, ensuring that one of the pair partners is new to the code • Ensuring that the tests that exist are run every day
Courage	Cannot proceed as aggressively with certain simple design	• Add at least one automated test every time a defect is fixed • When stories require changes to legacy code, increasing the amount of design discussion during the Iteration Planning Meeting

^a A Wiki is a Web-based collaborative repository popular with XP teams. See www.wiki.org for more information.

Value be diminished as a result? In the case of our untestable legacy code, a quick brainstorming session by the team might yield the ideas in the Impact column of Table 3. The team discusses ways to adapt the process that is guided by the values, yielding something similar to the Adaptation Alternatives column.

Each alternative that the team considers is checked for its alignment to the values. A misaligned example, an alternative that states "all legacy code changes must be approved by a Change Control Board (CCB) prior to implementation," may be viable, but it is not simple to implement. It reduces the frequency of feedback while we wait for the CCB to meet, and takes empowerment away from the programmers, thus reducing their Courage. Other alternatives that address the constraint and that align closer to the XP Values are preferred.

Using this simple technique, teams adapt the XP Practices to their project and team needs. The importance of starting with Courage cannot be overstated. Many teams have been able to achieve a level of simplicity in their practices beyond what was thought possible. Although this may appear to introduce risk, Retrospectives after each iteration mitigate that risk by helping the team understand where additional adaptations are required.

SUMMARY

The pace of change in the software development industry remains at high. People continue to push the boundaries of known techniques and practices in an effort to develop software as efficiently and effectively as possible. Extreme Programming and Agile Software Methodolo-

gies have emerged as an alternative to comprehensive methods designed primarily for very large projects. Teams using XP are delivering software often and with very low defect rates. As the industry continues to evolve, we are likely to see additional insights on how to leverage collaborative work on more and more types of projects. ▲

Notes

1. Beck, K., *Extreme Programming Explained*. Addison Wesley Longman, 2000.
2. www.agilemanifesto.org.
3. Sliwa, C., "Agile Programming Techniques Spark Interest," *Computerworld.com*, March 14, 2002.
4. Jeffries, R. et al., *Extreme Programming Installed*, Addison Wesley Longman, 2001, 172.
5. It is likely true that skilled practitioners of most methods are guided by a set of values, perhaps dictated by the culture of the organization, perhaps dictated by the leadership of the team. It is the expression of the values as integral to the practices that makes XP unique.
6. Schwaber, K. and Beedle, M., *Agile Software Development with Scrum*, Prentice Hall, 2002.
7. Mary Poppendieck, *Lean Development: A Toolkit for Software Development Managers*. Addison-Wesley, to be published in April 2003.
8. Thomsett, R., *Radical Project Management*, Prentice Hall, 2002.
9. Fowler, M. et al., *Refactoring: Improving the Design of Existing Code*, Addison-Wesley, 1999.
10. For more information on good software design principles and their application on agile software projects, see Martin, R.C., *Agile Software Development: Principles, Patterns, and Practices*, Pearson Education, 2003.
11. Kerth, N., *Project Retrospectives*, Dorset House, 2001.

COMPONENT-BASED IS ARCHITECTURE

Les Waguespack and William T. Schiano

Advances in distributed architectures and the Internet have had a strong influence on the use of component technologies. The authors discuss the organizational and technical requirements for an effective and economically viable component-based approach.

A COMPONENT IS A BUILDING BLOCK OF computer software systems. Modular construction has been the standard of software architecture for decades. What makes components intriguing now is the integration of technologies to support components, the use of components to compose business applications rather than the underlying system software, and component deployment and distribution over the Internet. These factors converge, leading to the prospect of shortened application development time and increased application software quality.

Problem decomposition is a fundamental technique in systems analysis. Through the process of decomposing the whole, one discovers the units of information and function that must come together to define and achieve the behavior of the whole. This guiding principle permeates architecture and civil, electrical, and electronic engineering. It is the same in software engineering employed in the construction of computer-based information systems.

Component technology is an outgrowth of the object-oriented paradigm of system modeling, and it mirrors the technique of problem decomposition by encouraging system composition using a collection of interacting, but independently constructed parts. In this context, *component* takes on the specialized meaning for which we use the term in this chapter.

This article discusses the definition, supporting technology, use, construction, and economics of components. The goal is to explain how building components and building

applications using components differ from traditional software development practice. Components have the potential to shorten application development time and reduce overall development costs. To capitalize on this potential, organizations must prepare their processes and personnel for component-based application building. For the component-based approach to be cost-effective, organizations must identify requirements for componentization with a sufficiently broad internal or external market for use and reuse. The supply and demand side issues interact in a complex economic viability model that extends beyond organizational boundaries.

DEFINING COMPONENTS

The industry has not yet settled on a universally accepted definition of "component." In the broadest sense, a component is an artifact of systems development manufactured explicitly for the purpose of being used in the construction of multiple systems by multiple development groups. This definition encompasses most knowledge or artifacts reused in system building. That could range from documentation standards and templates to library subroutines and programming languages. How a component is used is a better way to define it than how it is built. What a component is depends on how it is used.

This article focuses on the role of components as building blocks in business applications — *business domain components*. We

LES WAGUESPACK is a professor of CIS at Bentley College and former systems architect in the United States Air Force. He can be reached at lwaguespack@bentley.edu.

WILLIAM T. SCHIANO is an associate professor of CIS at Bentley College and former president of thoughtbubble productions. He can be reached at wschiano@bentley.edu.

Application
development
becomes a
construction
set exercise.

choose this focus to exclude building blocks of systems software (e.g., device drivers, graphics tool sets, or database management packages). The stakeholders of system software components are hardware and operating system vendors whose concern is platform efficiency and interoperability. The stakeholders of business domain components are business users, managers, and systems analysts striving to satisfy requirements driven by business processes, business markets, and government regulations. We use the term "component" to mean a business domain component for the remainder of this chapter.

A component refers to an element of software that is clearly defined and separable from the system. It interacts with the rest of the system through an explicitly defined interface. Except for the interface, a user's knowledge of the component's internals is unnecessary for it to properly function in the rest of the system. Recent advances in software system architecture have enabled greater dispersion of components on networks and on the Internet and have therefore been catalysts for independent component development and their interchangeable combination. A component can be used in myriad system constructions.

Components marketed by out-of-house (third-party) producers are called *off-the-shelf components*. If a system developer can locate and use an off-the-shelf component, the cost of building a new, perhaps a one-time-only, solution for a particular requirement can be avoided. Off-the-shelf components arrive packaged, documented, and tested; they pose a cost-saving alternative to developing new software for each new system requirement.

Components Based on Business Requirements

Application systems for a particular organizational function (or cross-functional process) often share a variety of very similar computational requirements. Domain analysts recognize these similar requirements as opportunities to define components. Architects can use components during requirements analysis to frame an application domain. Components are stable architectural primitives to be combined or arranged to suit a business procedure or process. When the procedure or process changes, components can be rearranged quickly and easily without having to revisit the component definition. Systems analysts can use components as a lexicon for expressing individual application

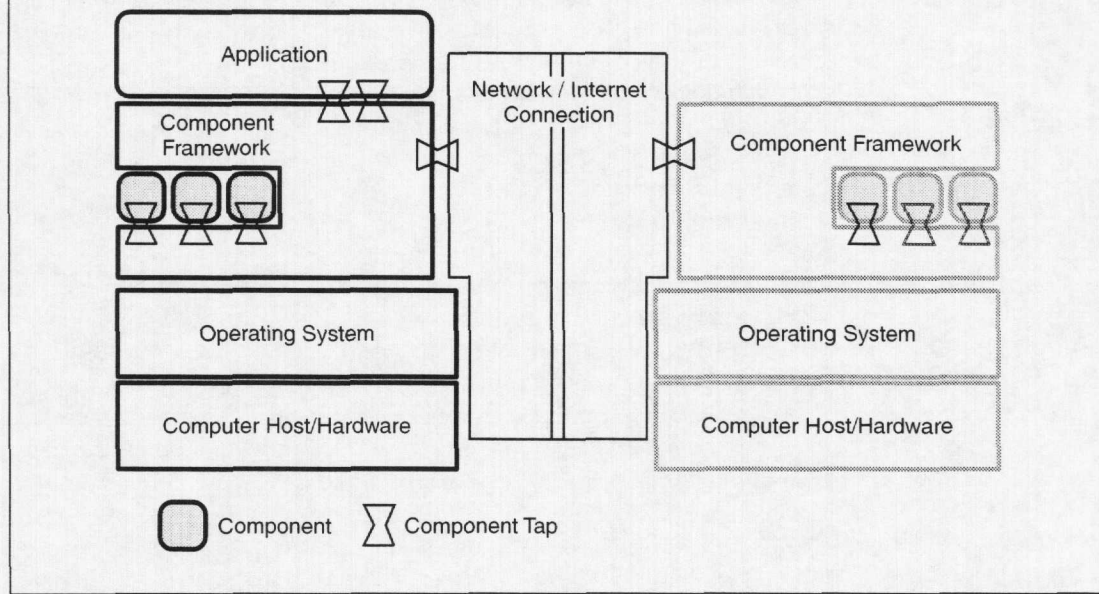
requirements. Designers can use components as predefined parts around which to structure a system. Programmers can use components as tangible building blocks to construct executable software.

At each step of software development, a component represents an explicit understanding within the application domain: what information is germane, what actions are relevant, how action is triggered, and what consequence an action has. This constitutes a contract among the members of the extended development team — the component is a stable, reliable, known quantity. Honoring the component definitions and interfaces obviates understanding their inner workings. Developers are free to ignore details other than those pertaining to business domain function to the extent that the collection of components represents a complete lexicon of the application domain. Application development becomes a *construction set* exercise.

Component Enabling Technology

Today's mainframe, desktop, portable, and handheld computers are more easily interconnected than ever before. Because this connectivity is virtually continuous, systems architects often think of their systems as network based or Internet based rather than computer based. If a resource exists anywhere on the network-based system, it is readily accessible by other computers on that network. This *distributed computing* model is fertile ground for components. As an example, an application designer may need some computation (e.g., credit card validation, decryption, calendar arithmetic, format translation, etc.). If the designer can find a suitable service on the network, engaging that service rather than programming it from scratch may result in cost savings.

The technology that enables components is the *component framework*. A component framework is a combination of protocols and system software that resides on both the component server and client computer (see Figure 1). The framework handles the interface connections between an application and a component. The component resides on the server computer. The application (component user) resides on the client computer. They may be the same or different computers on a network. The framework hides as many details about the component implementation, the host operating system and hardware, and the network connection as possible. A component framework

FIGURE 1 Component Architecture

provides a communication path between the application and component. Regardless of the surrounding environment, the application can ignore how the component is implemented or even where it is located, and vice versa.

A component framework achieves component/application independence by providing a set of component services relating to naming, event handling, transactions, persistence, and security. These services combine to provide a transaction processing environment. Component naming services permit applications and components to identify and locate one another. Once they establish communication, they exchange information about their capabilities, services, and information formats. Component event services enable applications and components to get each other's attention and synchronize actions. Transaction services define collections of application activity that require all-or-nothing completion, regardless of local or remote component interaction. Component persistence services allow components to perform database management activities. Component security services, coupled with naming services, provide for client/server authentication and control access to both information objects and component functionality. All the services described herein are found to some degree in every component framework, but the developer's (or vendor's) styles and approaches differ widely.

Component Framework Products

This section presents a brief survey of component frameworks. We choose these frameworks to illustrate the evolution of standards and product features over the past decade or so.

- **CORBA.** Common Object Request Broker Architecture (CORBA) is a component framework standard promulgated by the Object Management Group (OMG), a consortium of major computing industry players. CORBA defines protocols, services, and an execution environment for components. The OMG does not market components or a component framework. It enforces the CORBA standard and provides test suites to certify CORBA compliance. CORBA compliance assures component producers, component framework producers, and their customers that applications and components will interact reliably and consistently regardless of the vendor. The OMG aggressively solicits suggestions for extensions and improvements to the CORBA standard and publishes a wide variety of supporting technical documentation and training materials. The CORBA standard has been instrumental in enabling the cross-vendor, out-of-house component market.
- **Microsoft ActiveX.** Microsoft has defined four standards along the way with a suite of products it markets to support components. Microsoft's industry clout has given these standards some prominence. ActiveX predates

The configuration manager's task is sorely compounded when applications or systems include dozens or even hundreds of components.

Microsoft's other component enabling tools. ActiveX uses a Web browser on a client machine to download an ActiveX module. Once downloaded, it executes directly on the client's underlying MS Windows machine. ActiveX might be described as "just-in-time downloaded programs."

- **Microsoft DCOM.** Distributed Component Object Model (DCOM) is an inter-object communications protocol designed to allow components on various nodes of a Microsoft network to interact. It extends the functionality of the earlier standard, COM, which provided similar function, but on a single host computer. Tightly coupled with Microsoft's operating system product line, MS Windows, COM, and DCOM support a thriving market in components for MS Windows applications, including MS Office, Visual Basic, Visual C++, and J++.
- **Microsoft .NET.** Microsoft's most recent offering is .NET. It extends DCOM, adding a robust component development and configuration environment. It provides application-to-component connectivity, regardless of their location on a computer, a network, or the Internet. The .NET product suite includes program editing, compiling, and debugging in an integrated environment, Visual Studio. Several .NET-compliant programming languages are available to develop both applications and components.
- **Enterprise JavaBeans and J2EE.** Another major player in component frameworks is Sun Microsystems. With its invention of Java, Sun pioneered the "just-in-time downloaded programs" not only for Web browsers on MS Windows, but for any Web browser with a compliant Java virtual machine (JVM). JVMs are available for most contemporary operating systems with or without Web browsers. Unlike ActiveX, which is executable object code for MS Windows-based computers, Java compiles into machine-independent byte-code. Byte-code then executes interpretively on a JVM on the client machine. Java, unlike ActiveX, is both machine and operating system independent; and because it is interpreted, it does not pose the security problems that downloaded executable object code presents. Java is a popular choice of developers who need "program once" and "run anywhere" software. Sun Microsystems builds upon its Java base with Enterprise Java Beans and J2EE that support a program editing, compiling, and debugging environment for Java-based components.

J2EE and .NET represent the state-of-the-art in Internet-enabled, distributed component architecture technology. Each represents the evolution of component support from simply enabling components to connect and communicate, to an insulating enclosure of development, communication, database, and process management services that thoroughly enables (enforces) a model of scalable, distributed enterprise computing.

BUILDING APPLICATIONS USING COMPONENTS

Building applications using components differs from traditional application software development practice where programmers build information systems composed of tailor-made, one-of-a-kind, built-from-scratch programs. Effective use of components to build a new system requires a development process that is *component aware*. Component awareness refers to an approach in which a high value (within the organization) is placed on using components rather than building software from scratch, for (1) identifying recurring requirements suitable for component solutions and (2) streamlining processes to make component use efficient. Regardless of the specific steps or the specific sequence in the software process, each step requires component awareness. Either all the personnel involved in the application building process must become individually component aware, or specific personnel are assigned to oversee the use of components, or a combination of both.

In a component-aware software process, requirements analysis identifies units of required functionality that are likely to be available as off-the-shelf components. Systems designers are familiar with the catalog of available off-the-shelf components relevant to their application domain. Programmers are familiar with the component framework environment into which the components will plug. Programmers are able to configure their noncomponent system pieces to interoperate with that environment as well. Component-aware projects present a new level of challenge for configuration managers. Warranty, availability, licensing, liability, maintainability, and serviceability are all familiar issues with outsourced software or services. The configuration manager's task is, however, sorely compounded when applications or systems include dozens or even hundreds of components.

TABLE 1 Component Reusability Assessment Checklist

- What does it do?
 - Is it designed for our application domain or do we have to adapt it?
 - Does it support application domain functionality standards, ISO, FASB, IRS, FDA, etc.?
- Who provides it?
 - Does it come from in-house or a third party?
 - What form of warranty or guarantee does the provider offer?
 - Is the provider organizationally stable/reliable?
 - Does the provider offer configuration support services or consulting?
- What are its environmental requirements?
 - Does it require a specific component framework environment (e.g., CORBA, DCOM, J2EE, .NET, etc.)?
 - Does it require a specific configuration tool environment (e.g., Visual Studio, JavaBeans, etc.)?
 - Does it require specific programming language expertise (e.g., C++, Java, VB, etc.)?
 - Is it compatible with other components we may be considering for use with it?
- How is it used?
 - Is it configurable (i.e., parameters, extension points, variable through inheritance, etc.)?
 - Does it generate source code requiring translation or linkage?
 - Does it run autonomously as a service accessed dynamically via a network or the Internet?
 - Are there multiple versions of the component, and which one is best for this project?

Each component considered, whether it is used in a project or not, exacts a cost. At a minimum, there is the cost of learning enough about the component to determine whether it is useful or not and, then, whether or not it is feasible to use it. The checklist above (see Table 1) is representative of the information needed to consider a component for use in a particular development project. The questions show a progression of detailed knowledge needed as the project edges closer to actually using a component.

Finding candidate components and then learning the answers to these questions may represent a significant expense. To maximize the return on investment, an organization should establish its own internal catalog of the components it has examined, where they have been used, who used them, and what has been learned about them to date. The internal catalog serves as the first point of search when future component requirements arise. It is worth noting that the internal catalog will likely contain a great deal of information about components that have not yet, and may never, be used.

To control the costs of finding components and assessing their potential for any given requirement, the system's project tasks and software development life cycle need to include an explicit component information collection and management activity. These changes in organizational practice are not confined to a small group of workers, but should be widely integrated into the organization's practices. Such a broad integration relies on a strong management commitment and a disciplined project management approach. It is likely that organizations that are

only casual followers of formal software engineering practices will find the task of formally maintaining a *component-aware* organization very difficult.

BUILDING COMPONENTS

Just as there are organizational challenges to becoming an effective consumer of components, there are also many challenges to becoming an effective producer of components. Components are intrinsically different from application software. Unlike software written as a part of a larger product, components are intended to stand alone as products themselves. They require their own life-cycle management, testing, documentation, and support. If they are destined for consumers outside the producer's organization, they require more sophisticated documentation, examples using them, user guides, and customer support.

Choosing which component enabling technologies to use is another challenge to component producers. (See "Component Enabling Technology" earlier in this chapter.) By analogy, should we build a toy car part using LEGO bricks or Tinker Toys? The choice affects not only the pool of potential consumers and the production costs, but also has architectural and compatibility implications on the component's longevity.

An even more difficult set of choices is the selection of candidate functionality for prospective components. As discussed in the "Defining Components" section in this chapter, there are few limitations on what can be conceived of or defined as a component: *a component is an artifact of systems development*

The degree of utility depends on the domain of functionality that is a consumer's focus.

manufactured explicitly for the purpose of being used in the construction of multiple systems by multiple development groups. Which components should be built? Should we build them? Intuitively, a component should have a high probability of being used and reused — that is, high *reusability*.

Component reusability results from three characteristics briefly described below:

1. *Utility*: A component's function must be relevant to a problem domain. It is common to find components that support a particular genre of system functionality (e.g., GUI services: windows, menus, dialog boxes). It is less common to find components that are application domain focused (e.g., account, client, policy, contract, agreement, etc.). Therefore, the degree of utility depends on the domain of functionality that is a consumer's focus. The greater the utility, the greater the reusability. (The component does something valuable.)
2. *Capacity*: A component's function must be sophisticated enough such that using the component is clearly advantageous compared to build-from-scratch development. Searching, finding, learning, and then using a component is a labor-intensive effort. It would seem that good component candidates would have some degree of complexity in their functionality, along with the testing that would certify reliable and, perhaps, efficient performance. The greater the capacity, the greater the reusability. (What the component does is difficult to build from scratch.)
3. *Versatility*: A component's implementation must permit convenient integration into a target application's structure. Unless a component can be applied in a host application "as is," some configuration or adaptation is required for one, the other, or both. Adaptations become the maintenance and support responsibility of the component consumer, and these costs may outweigh the reusability benefits of the component. In the extreme, "It may be more trouble than it's worth!" The greater the versatility, the greater the reusability. (Although the component does not do exactly what is needed or how it is needed, it is easy enough to adapt for the need at hand.)

An organization's success in achieving reusability depends heavily on the producer's understanding of the consumer's problem domain. Utility depends on the problem domain almost

exclusively. Capacity depends on an understanding of the architectural nature of the problem domain — which requirements are permanent and which requirements are evolving. Permanent requirements would seem to present component candidates with greater longevity potential. Versatility is strongly influenced by the choice of implementation technology and the degree of variability that a particular component must support — as in the case of evolutionary requirements.

Reusability forms a basis for a cost-benefit analysis to guide the selection of requirement candidates to implement as components. When the producer is also the consumer, an activity called *domain analysis* needs to be performed. In domain analysis, the component producer attempts to construct a model that abstracts the requirements of prospective consumers of the component in an attempt to identify shared functionality requirements.

In the final analysis, it boils down to questions of economics: the consumer asks, "Does building systems using components increase or decrease the system life-cycle costs?" And the producer asks, "Does the economic return on components produced justify the cost of building them?" These questions are explored in the following section.

MAKING COMPONENTS COST-EFFECTIVE

Components might streamline systems but using them increases the complexity of the development process. Whether an organization is exclusively a consumer of components, a builder of components, or both, all aspects of their software development life cycle are affected.

Component Consumer Issues

First consider the component consumer and, for the sake of discussion, assume that useful components are readily available. Project management of component-based initiatives involves new challenges. Projects need to integrate a component culture throughout the software development life cycle in requirements specification, design, programming, testing, and maintenance. A third party may control the life cycle of some of a project's components.

In requirements specification, the scope becomes broader, as developers need familiarity with not only their requirements, but also what components are available and the interoperability frameworks on which they depend.

While a component-based project may produce fewer lines of original programming, it must address an expanded number of management issues.

Components may work with one framework but not another.

In design, developers must choose whether to adapt the design to accommodate an off-the-shelf component, develop software to adapt the component to the requirement at hand, or forego the use of a component and just write code. Developing software to adapt the component may not be cost-effective for a single instance; but if the component is useful in other, sufficiently numerous circumstances, the adaptation cost may be justified. Such a judgment requires thorough knowledge of the requirements gained through domain analysis, which itself represents a new discipline for many, and will require learning.

Maintenance becomes more involved because of the myriad interactions. Requirements may change in one instance of component use, but not in others. A change in one component may require changes in interfaces to software modules or other components and the resulting testing. Components need to be certified as reliable and efficient to save a consumer's effort. Component providers need to demonstrate that they or a third party have certified the component. Unless the consumer can trust the component's reliability, the consumer must perform certification. When components are certified, testing is limited to how they are configured and integrated and excludes their inner workings.

For the sake of the above discussion, it is assumed that components were readily available. However, components may not be readily available for many development projects — from either producers or external vendors.

Component Producer Issues

Component producers must decide which components to produce. Producers need domain analysis to assess a component's efficacy. Rather than satisfying a requirement from a single consumer's perspective, the component producer attempts to divine a utility with the prospect of repeated use by, perhaps, several consumers in many similar requirement situations. Choices must be made about the level of granularity and interoperability.

Specialized components with complex functionality may limit the number of potential consumers. Fewer consumers across which to amortize the development cost means higher per-consumer cost to acquire a component. Higher component acquisition cost eats into the potential life-cycle savings of choosing a

component over build-from-scratch, and so on. Component function granularity is a delicate design parameter for the component producer.

Consumer access to a component depends on which interoperability framework the producer chooses for it. The producer must follow the framework market and assess not only the current capabilities, but also the future capabilities that each framework vendor might be contemplating. These decisions must be revisited each time a component is revised or upgraded.

The testing task of component producers is difficult. Although components may have compact and well-defined interfaces, testing must be extremely thorough and perfectly consistent with the component documentation. A major determinant in the component consumer's adoption decision lies in the prospect of cost savings attributable to the component's reliability. From the developer's perspective, this is a major challenge because testing must often occur independent of the eventual installation. Although not commonplace at this writing, third-party certification of compatibility and reliability of components is inevitable as the component market evolves.

Documentation extends to configuration and to the domain, which increases the scope and expense of the documentation process. The wider the range of users of the documentation, the greater the importance and the required sophistication of the documentation. Components must be cataloged in a way that enables consumers to find and evaluate them.

Shared Producer and Consumer Issues

To achieve a significantly reduced time-to-market, a great deal of development must take place in parallel, requiring coordination among groups. While a component-based project may produce fewer lines of original programming, it must address an expanded number of management issues. Reducing the effort expended in traditional development with component reuse requires disciplined and effective management of the development, selection, and use of components. Key to that is cost accounting.

Cost accounting is difficult for all IS organizations, and many ignore it or use only rough heuristics. Component reuse introduces additional project costs, including warehousing, delivery, and use costs. Those costs affect each project that uses the component and raise the required initial investment. Any cost savings from reuse accrue back to the component.

From the purchase of packaged software through full-service outsourcing of entire departments, outsourcing is a routine part of many information systems (IS) organizations. Similarly, when deciding whether to build or buy components, management must determine if the savings from reuse will exceed the incremental cost of obtaining a reusable component. Typically, savings occur only when a component is reused several times. These reuses often occur over several years, introducing significant time value of money issues. To be cost-effective, organizations must obtain reusable components that maximize the opportunity for repeated instances of reuse and limit the need for writing code.

If the organization finds that acquiring a component is the best option, it must still choose where to buy it. Vendor selection processes involve significant investment on the part of the purchasing organization and the selling organization. The benefits of such screening are well established but it is also important to make sure that the costs of the evaluation process do not become too high in the case of inexpensive components.

There are several viable pricing models for components. Components can be sold to out-

side consumers, who can then incorporate them into their own systems. They can also be licensed under a variety of terms, including time period, volume, per application, and per execution. Given that components can run on remote servers owned by the developer, licensing based on usage may become more common.

CONCLUSION

Components encompass many widely accepted traditional software engineering principles implemented with new technologies. The emergence of the standards and frameworks described in this chapter, along with the support of the OMG, reflect the increasing maturity of the process of component development and use. The markets for components have been much slower to evolve. While there are many valuable generic components available, domain-specific components remain in short supply. Until more organizations start to use components, most domain-specific markets will be too small to support third-party development. Organizations considering wide-scale implementation of components will likely need to develop some components in-house. ▲